

Министерство науки и высшего образования Российской Федерации

МГТУ им. Н. Э. Баумана

Кафедра ИУ-5 «Системы обработки информации и управления»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к итоговому проекту

«JobAbility»

Система трудоустройства для людей с ограниченными возможностями

Студент: Илья

Группа: _____

Преподаватель: _____

Москва, 2026

Введение

Актуальность проекта JobAbility связана с тем, что цифровой рынок труда должен учитывать не только поиск вакансий по профессии и зарплате, но и реальные условия доступности рабочего места. По данным Всемирной организации здравоохранения, около 1,3 млрд человек, то есть примерно 16 % населения мира, живут со значительными ограничениями здоровья. Поэтому задача инклюзивного трудоустройства является массовой социальной и технологической задачей, а не узкой вспомогательной функцией кадрового сервиса [1].

В российском правовом поле занятость инвалидов регулируется Федеральным законом от 24.11.1995 № 181-ФЗ, в котором закреплены меры социальной защиты, квотирование рабочих мест и необходимость специальных условий труда [3]. На практике это означает, что работодатель должен не просто разместить вакансию, а описать график, формат работы, доступность офиса, возможность удаленной занятости, требования к коммуникации и поддержку ассистивных технологий.

Цель работы — разработать и развернуть веб-систему JobAbility для публикации доступных вакансий, регистрации соискателей и работодателей, формирования заявок, модерации вакансий и заявок, хранения медиафайлов и автоматического обновления приложения через CI/CD в Kubernetes.

Назначение системы состоит в том, чтобы объединить в одном информационном контуре трех участников: соискателя, которому нужен понятный каталог и безопасная заявка; работодателя, которому нужен инструмент публикации вакансий; модератора, который контролирует качество данных и завершает обработку заявок.

К нефункциональным требованиям относятся: адаптивность интерфейса на desktop и mobile; ролевая модель доступа; хранение структурированных данных в PostgreSQL, медиафайлов в MinIO и временных данных в Redis; воспроизводимое развертывание в Docker/k3s; автоматическая сборка, тестирование и доставка через GitLab CI/CD;

наблюдаемость через Prometheus и Grafana; приемлемое время ответа при демонстрационной нагрузке до 10 RPS.

Задачи работы включали анализ предметной области, проектирование бизнес-процессов и диаграмм, реализацию backend на Django REST Framework, frontend на React, Dockerfile для backend и frontend, Kubernetes-манифесты для всех сервисов, настройку GitLab Runner, подготовку pipeline build-test-upload-deploy, заполнение демонстрационных данных и оформление РПЗ.

Предметная область

Предметная область рассматривается на примере условной компании «JobAbility Partners» и ее подразделения «Центр инклюзивного найма». В процессе участвуют HR-менеджер по инклюзивному найму, специалист по адаптации рабочих мест, модератор платформы и руководитель отдела подбора. HR-менеджер создает вакансии, специалист уточняет условия доступности, модератор проверяет корректность описаний, а руководитель оценивает отклики и результативность подбора.

В терминологии проекта соискатель (candidate, applicant) — пользователь, который ищет работу и формирует заявку; вакансия (job vacancy, job posting) — описание рабочего места; работодатель (employer, company) — организация, публикующая вакансию; заявка (application, request) — набор выбранных вакансий и данных соискателя; профиль (candidate profile) — контактные и анкетные сведения пользователя. Эти понятия согласуются с терминологией исследований по inclusive employment и job matching [4].

В статье Kuznetsov et al. 2025 описана production-ready система инклюзивного подбора работы, где центральными сущностями являются candidate, company, job position, readiness и compatibility score. Цель исследования — показать, что ML-система подбора может быть встроена в реальный процесс службы занятости. Среди заявленных результатов: F1-

score 90,1 %, время ответа менее 100 мс и обработка 500 000 комбинаций candidate-company менее чем за 10 минут [4]. Для JobAbility из этой работы важен вывод: система должна хранить не только вакансию, но и признаки доступности рабочего места.

В работе Kamruzzaman и Kim 2025 рассматривается disability disclosure — раскрытие сведений об инвалидности кандидата — и влияние этого фактора на fairness in candidate selection. Авторы показывают риск смещения в LLM-driven hiring: при прочих равных условиях кандидаты, явно указавшие отсутствие инвалидности, могут получать преимущество [5]. Поэтому JobAbility не выполняет автоматический отказ по данным о здоровье: профиль используется для оформления заявки, а финальное решение в процессе оставлено человеку-модератору.

Для ручной аннотации терминов рекомендуется распечатать preprint arXiv:2508.11713. В нем удобно подписать перевод ключевых терминов: user — пользователь, candidate — соискатель, company — работодатель, job position — вакансия, application — заявка, accessibility/accommodation — доступность и адаптация рабочего места, matching — подбор.

Таблица 1 — Термины предметной области с переводом и источниками

Термин RU	English term	Назначение в JobAbility	Источник
Соискатель	candidate / applicant	Пользователь, выбирающий вакансии и формирующий заявку	[4], [5]
Работодатель	employer / company	Публикует вакансии и просматривает отклики	[4]
Вакансия	job vacancy / job posting	Рабочее место с зарплатой, графиком и условиями доступности	[4]
Заявка	application / request	Черновик или сформированное обращение соискателя	[4]
Раскрытие инвалидности	disability disclosure	Чувствительный признак, который нельзя использовать для дискриминации	[5]
Справедливость отбора	fairness in candidate selection	Требование не ухудшать положение кандидата из-за признака инвалидности	[5]

Ключевой расчет предметной области — расчет суммарной ожидаемой зарплаты по заявке. Если заявка содержит n выбранных вакансий, а в каждой строке указано количество позиций q_i и зарплата вакансии s_i , итог определяется формулой (1).

$$S = \sum_{i=1}^n (q_i \times s_i) \quad (1)$$

Для интерфейса также рассчитывается ориентировочная дата ответа. Она не является юридическим сроком, но помогает пользователю понимать ожидаемую длительность обработки заявки.

$$D_{\text{response}} = D_{\text{formed}} + \min(30, \max(1, n) \times 3) \text{ days} \quad (2)$$

Для оценки полноты описания вакансии используется показатель заполненности условий доступности рабочего места: отношение заполненных обязательных атрибутов к общему числу требуемых атрибутов.

$$C_{\text{access}} = (m_{\text{filled}} / m_{\text{required}}) \times 100\% \quad (3)$$

Патентный поиск и конкурентные решения

Таблица 2 — Результаты патентного поиска

№	Источник	Суть решения	Релевантность	Отличие JobAbility
1	Google Patents: US20140279637A1	Публикация по matching workflow между кандидатами и вакансиями	Подтверждает типовую задачу сопоставления профиля и вакансии	В JobAbility акцент сделан на доступности рабочих мест, заявке и модерации
2	Google Patents: search query «employment matching job seeker employer database»	Класс решений job matching с профилями пользователей и работодателей	Показывает зрелость рынка автоматизированного рекрутинга	Система не заменяет решение человека, а поддерживает процесс и хранит признаки доступности
3	Google Patents: search query «disability employment accommodation job matching»	Решения по подбору с учетом специальных условий и адаптации	Близко к предметной области инклюзивного найма	Реализован учебный web-сервис с открытым API, CI/CD и Kubernetes

Таблица 3 — Конкурентные решения

Решение	Сильные стороны	Ограничение относительно JobAbility	Что учтено в проекте
hh.ru	Большая база вакансий, фильтры, личные кабинеты	Нет фокуса на заявке для людей с ОВЗ и модерации условий доступности	Каталог, карточка вакансии, кабинет работодателя
Работа России	Государственный портал занятости, связь с мерами поддержки	Сложный универсальный сценарий, меньше учебной прозрачности API	Ролевая модель и социальная направленность
LinkedIn / Indeed	Международный поиск, профили, рекомендации	Инклюзивные признаки часто не являются центральной сущностью	Публичный каталог и профиль пользователя
AbilityJobs / GettingHired	Специализация на employment for people with disabilities	Зарубежный рынок и другая нормативная база	Фокус на доступности, безопасном отклике и человеческой модерации

Вывод по патентному и конкурентному анализу: JobAbility не претендует на уникальность идеи поиска работы как таковой. Отличие проекта состоит в учебной, но законченной реализации полного цикла: от публикации доступной вакансии до формирования заявки, модерации, хранения медиа, мониторинга и автоматической доставки в Kubernetes.

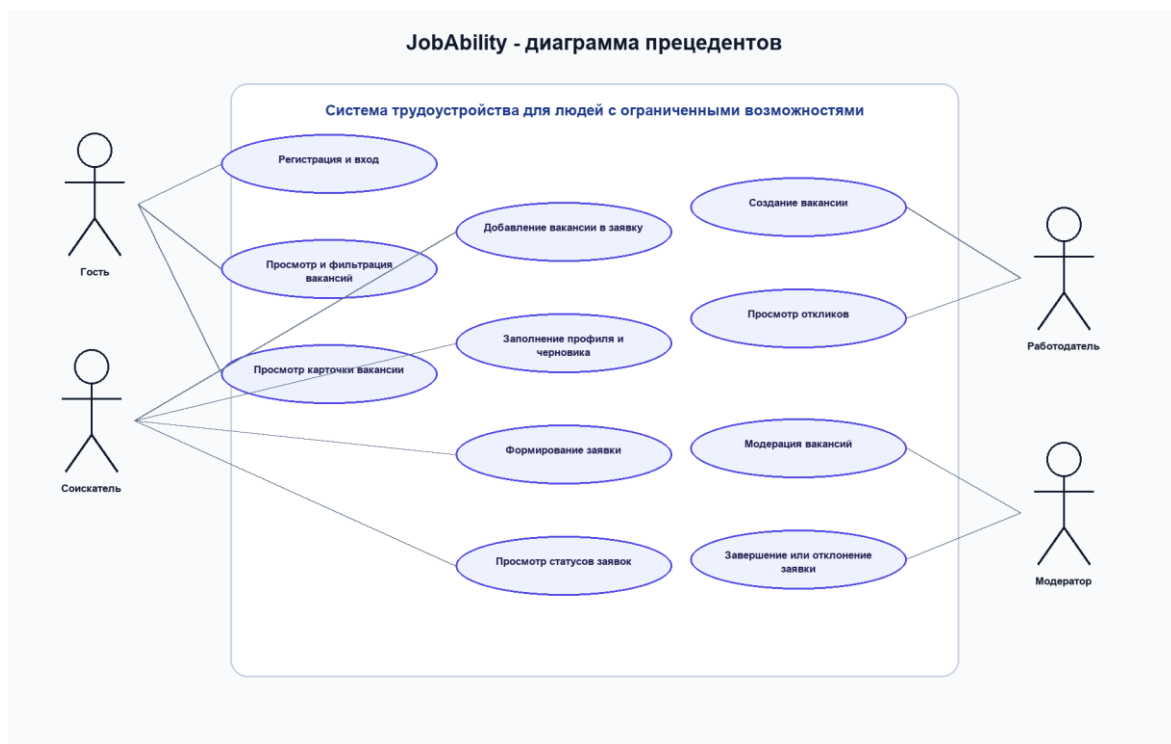


Рисунок 1 — Диаграмма прецедентов JobAbility

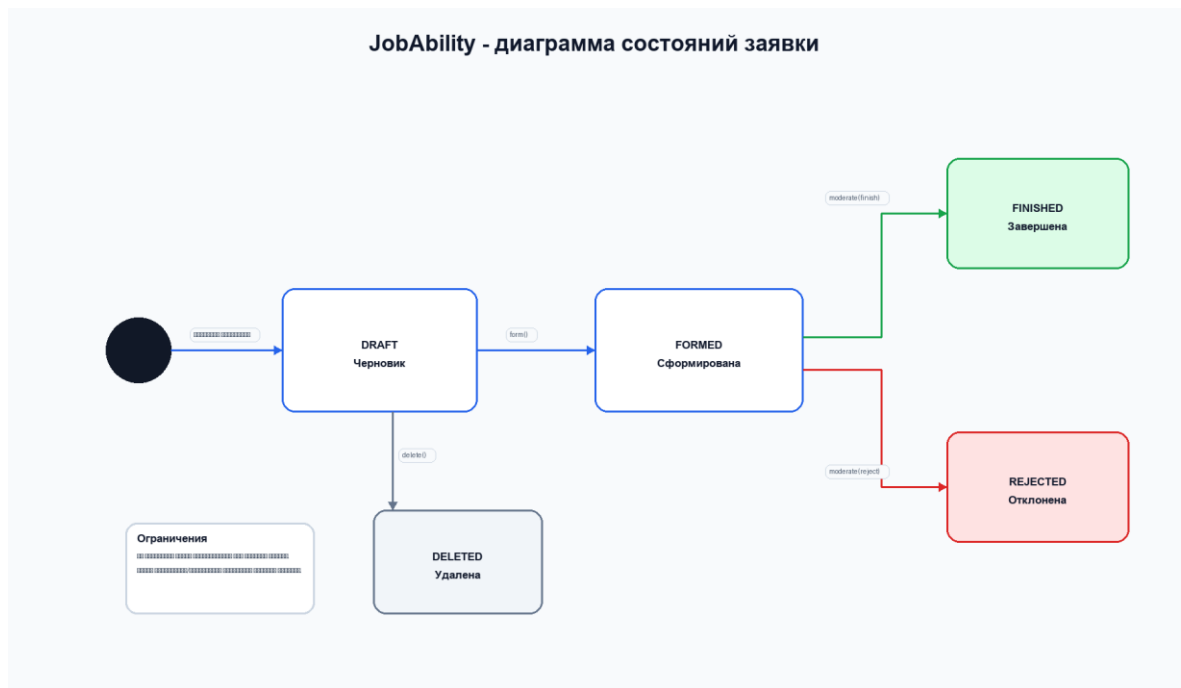


Рисунок 2 — Диаграмма состояний заявки

JobAbility - детализированный BPMN процесс трудоустройства

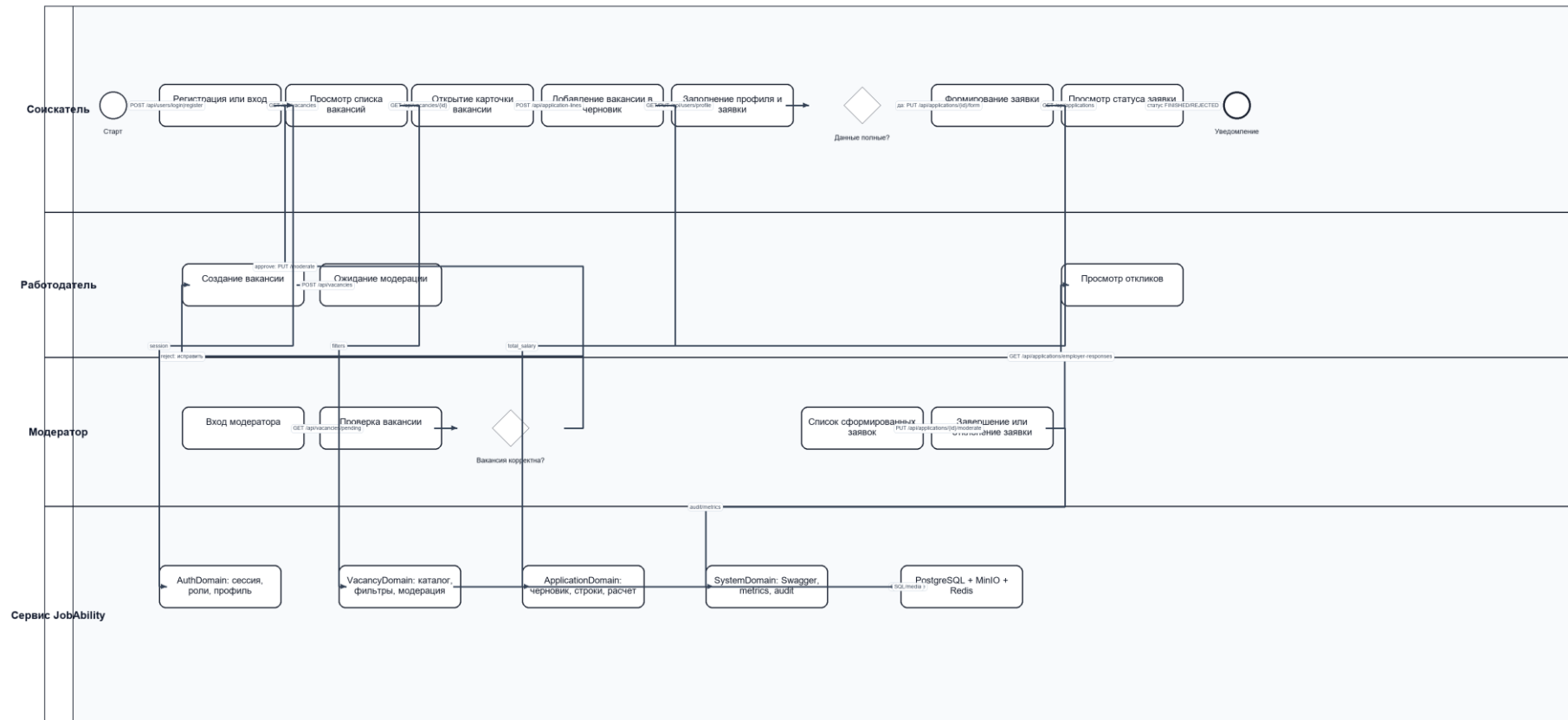


Рисунок 3 — Детализированная BPMN-диаграмма бизнес-процесса

Архитектура

Архитектура JobAbility построена как контейнеризованная web-система. Пользователь работает с React SPA, собранным Vite и размещенным в контейнере Nginx. Статический web-сервер отдает HTML, CSS и JavaScript, а запросы к /api, /admin, /swagger и /metrics передаются в backend-сервис.

Backend реализован на Django 5.2 и Django REST Framework. Он отвечает за домены Auth, Vacancy, Application, ApplicationLine, Profile, EmployerResponses и System. В backend сосредоточены правила ролей, проверка входных данных, расчет заявки, создание черновика, модерация вакансий и заявок, а также выдача OpenAPI/Swagger.

В Kubernetes приложение размещено в namespace jobability. Внутри кластера работают pods frontend, backend, postgres, redis, minio, prometheus, grafana и adminer. Traefik Ingress принимает HTTP-трафик на 80 порту и направляет его во frontend service. Backend обращается к PostgreSQL по TCP 5432, Redis по TCP 6379, MinIO по S3-compatible HTTP API на 9000 порту.

CI/CD реализован в GitLab. Pipeline содержит стадии build, test, upload и deploy. На build собираются Docker-образы backend и frontend, на test запускаются Django-тесты в backend-образе, на upload образы импортируются в containerd-хранилище k3s, на deploy применяются Kubernetes-манифесты и обновляются deployment/backend и deployment/frontend.

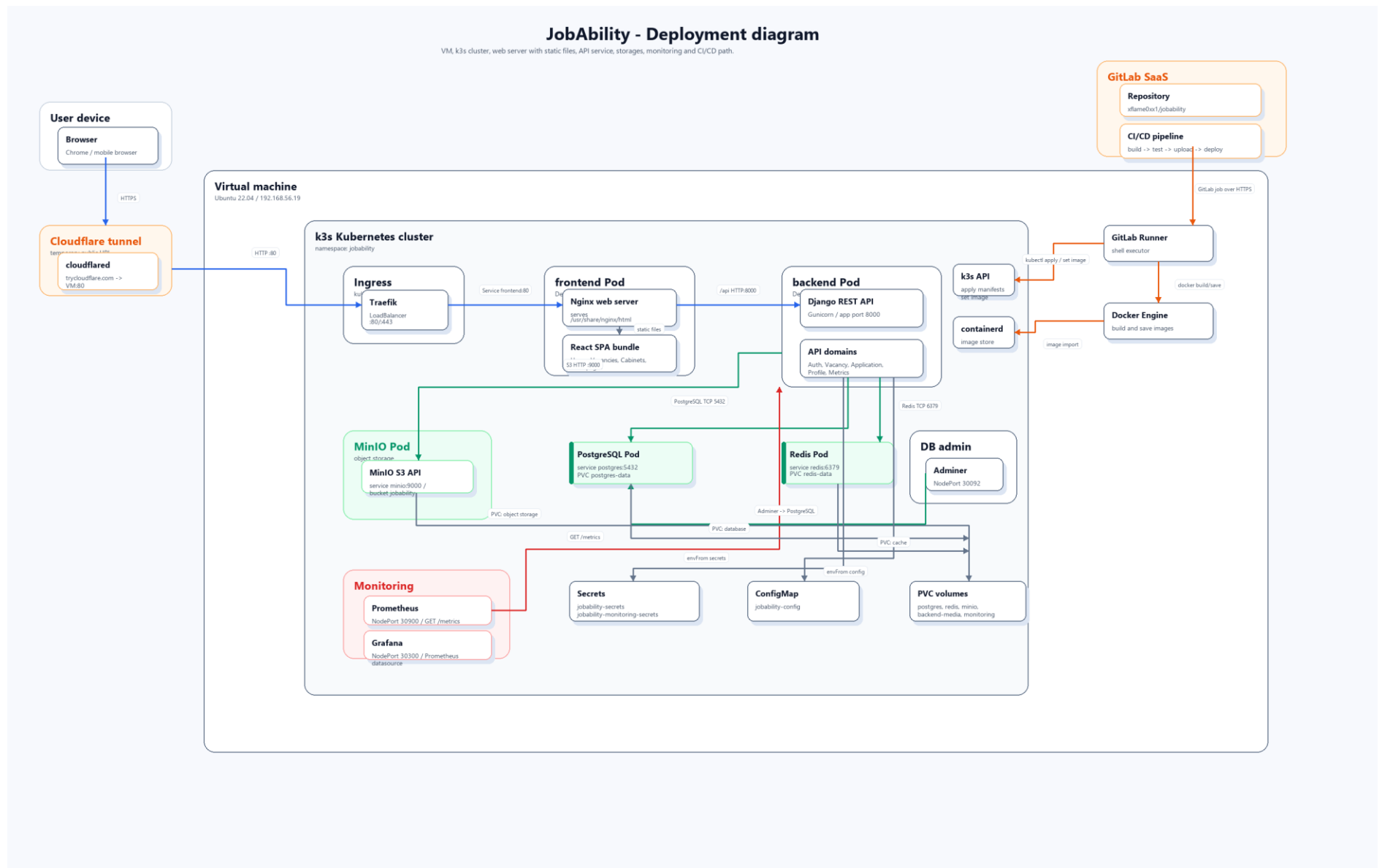


Рисунок 4 — Диаграмма разворачивания JobAbility



Рисунок 5 — ER-диаграмма основных таблиц

Таблица 4 — Назначение таблиц

Таблица	Назначение
auth_user	Учетные записи Django: логин, email, пароль и базовая идентификация пользователя
core_applicantprofile	Анкетные данные соискателя: ФИО, телефон, город, возраст, пол, категория инвалидности
core_vacancy	Вакансии работодателей: название, компания, город, зарплата, описание, график, поддержка доступности, изображение
core_application	Заявки соискателей со статусным циклом DRAFT → FORMED → FINISHED/REJECTED
core_applicationvacancy	Строки заявки, связывающие заявку с выбранными вакансиями и количеством позиций

Таблица 5 — Описание колонок основных таблиц

Таблица	Колонка	Тип	Описание
auth_user	id	int	Первичный ключ пользователя
auth_user	username, email	varchar	Логин и электронная почта
core_applicantprofile	full_name, phone, city	varchar	Контактные данные соискателя
core_applicantprofile	age, gender, disability_category	int/varchar	Анкетные сведения для оформления заявки
core_vacancy	title, company, city	varchar	Основные сведения о вакансии

core_vacancy	salary, schedule, disability_support	int/varchar	Зарплата, график и условия доступности
core_vacancy	image	varchar(100)	Имя/путь медиафайла, обслуживаемого через MinIO
core_application	status	varchar(16)	DRAFT, FORMED, FINISHED, REJECTED, DELETED
core_application	formed_at, completed_at	timestampz	Даты формирования и завершения заявки
core_application	total_salary	int	Итоговый расчет по строкам заявки
core_applicationvacancy	application_id, vacancy_id	bigint FK	Связь заявки и вакансии
core_applicationvacancy	qty, comment, is_main	int/text/bool	Количество, комментарий и признак основной вакансии

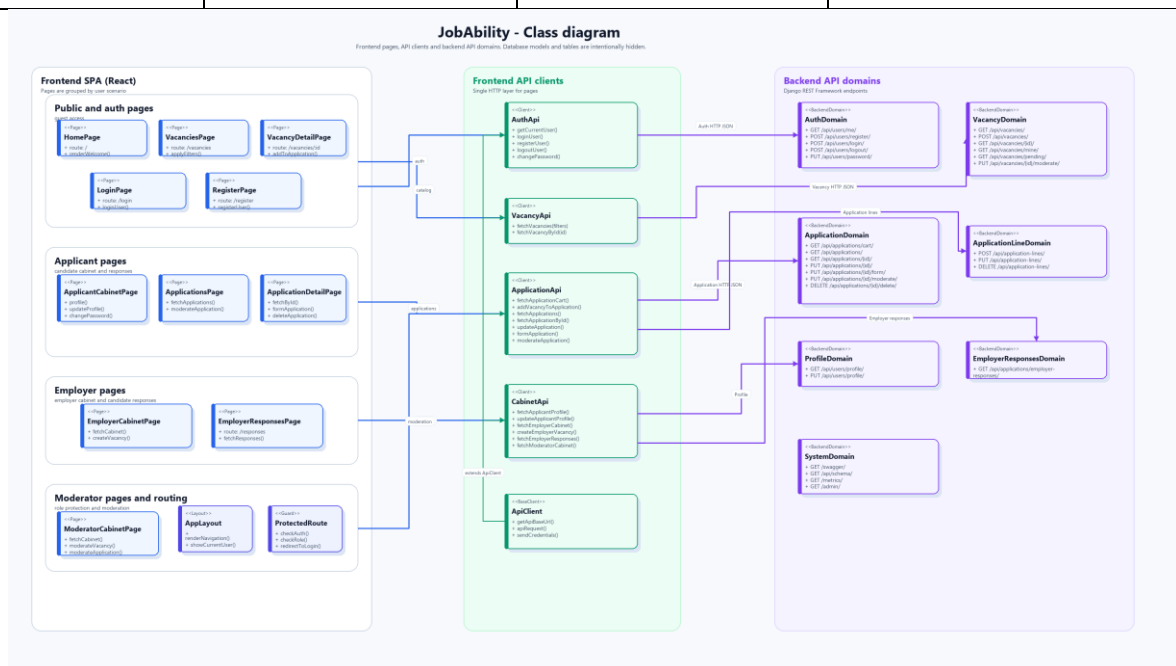


Рисунок 6 — Диаграмма классов frontend/backend доменов

Расчет аппаратных требований выполнен для 1000 зарегистрированных пользователей, пиковой активности 5 % и средней интенсивности 0,2 запроса в секунду на активного пользователя.

$$\begin{aligned}
 \text{RPS}_{\text{peak}} &= U \times k_{\text{active}} \times r_{\text{user}} \\
 &= 1000 \times 0.05 \times 0.2 = 10 \\
 &\text{requests/s}
 \end{aligned}
 \tag{4}$$

При расчетной стоимости backend-запроса 25 мс CPU и целевой утилизации 65 % минимальное количество ядер определяется формулой (5).

$$\begin{aligned} \text{CPU_cores} &= \text{ceil}(\text{RPS_peak} \times \\ \text{t_cpu} / \rho) &= \text{ceil}(10 \times 0.025 / \\ &0.65) = 1 \end{aligned} \quad (5)$$

С учетом PostgreSQL, Redis, MinIO, Prometheus, Grafana, k3s, containerd и резерва рекомендуется не менее 2 vCPU и 6-8 GiB RAM. Используемая виртуальная машина Ubuntu 22.04 с 7,1 GiB RAM и диском 23 GiB достаточна для демонстрационного контура.

$$\begin{aligned} \text{DB_year} &= \text{U} \cdot \text{B_user} + \text{V} \cdot \text{B_vacancy} \\ &+ \text{A} \cdot \text{B_application} + \text{L} \cdot \text{B_line} \end{aligned} \quad (6)$$

При 1000 пользователях, 500 вакансиях, 1500 заявках и среднем числе 3 строк на заявку расчетный объем PostgreSQL до индексов составляет около 13 MiB, а с коэффициентом 3 на индексы и служебные данные — 40-60 MiB. Основной рост ожидается в MinIO: 500 изображений по 300 KiB занимают около 150 MiB, поэтому объектное хранилище выделено отдельным PVC.

Алгоритмы

Основной алгоритм представлен последовательностью HTTP-запросов между браузером, frontend-страницами, API-клиентами и backend-доменами. Сначала пользователь выполняет POST /api/users/login/ или POST /api/users/register/. Backend проверяет данные, создает или открывает сессию и возвращает сведения о роли пользователя. После обновления страницы frontend вызывает GET /api/users/me/, чтобы восстановить состояние текущего пользователя.

Гость получает список вакансий через GET /api/vacancies/. В запрос могут передаваться search, min_price, max_price и другие фильтры. При открытии карточки выполняется GET /api/vacancies/{id}/. Если пользователь авторизован как соискатель, интерфейс дополнительно вызывает GET /api/applications/cart/. Если черновика нет, backend возвращает application_id = null и items_count = 0, что соответствует пустой иконке заявки.

Добавление вакансии в заявку выполняется POST /api/application-lines/ с vacancy_id и qty. Backend находит или создает черновую заявку DRAFT, добавляет строку и возвращает обновленные данные. Редактирование строки выполняется PUT /api/application-lines/ с vacancy_id, qty, comment, is_main и order_index; удаление строки — DELETE /api/application-lines/. Просмотр черновой заявки выполняется GET /api/applications/{id}/.

Формирование заявки выполняется PUT /api/applications/{id}/form/. Backend проверяет профиль, наличие строк, рассчитывает total_salary, фиксирует formed_at и переводит заявку в статус FORMED. Модератор получает список заявок через GET /api/applications/ с фильтрами, открывает заявку GET /api/applications/{id}/ и завершает ее PUT /api/applications/{id}/moderate/ с action = finish или reject. Работодатель получает список откликов через GET /api/applications/employer-responses/.

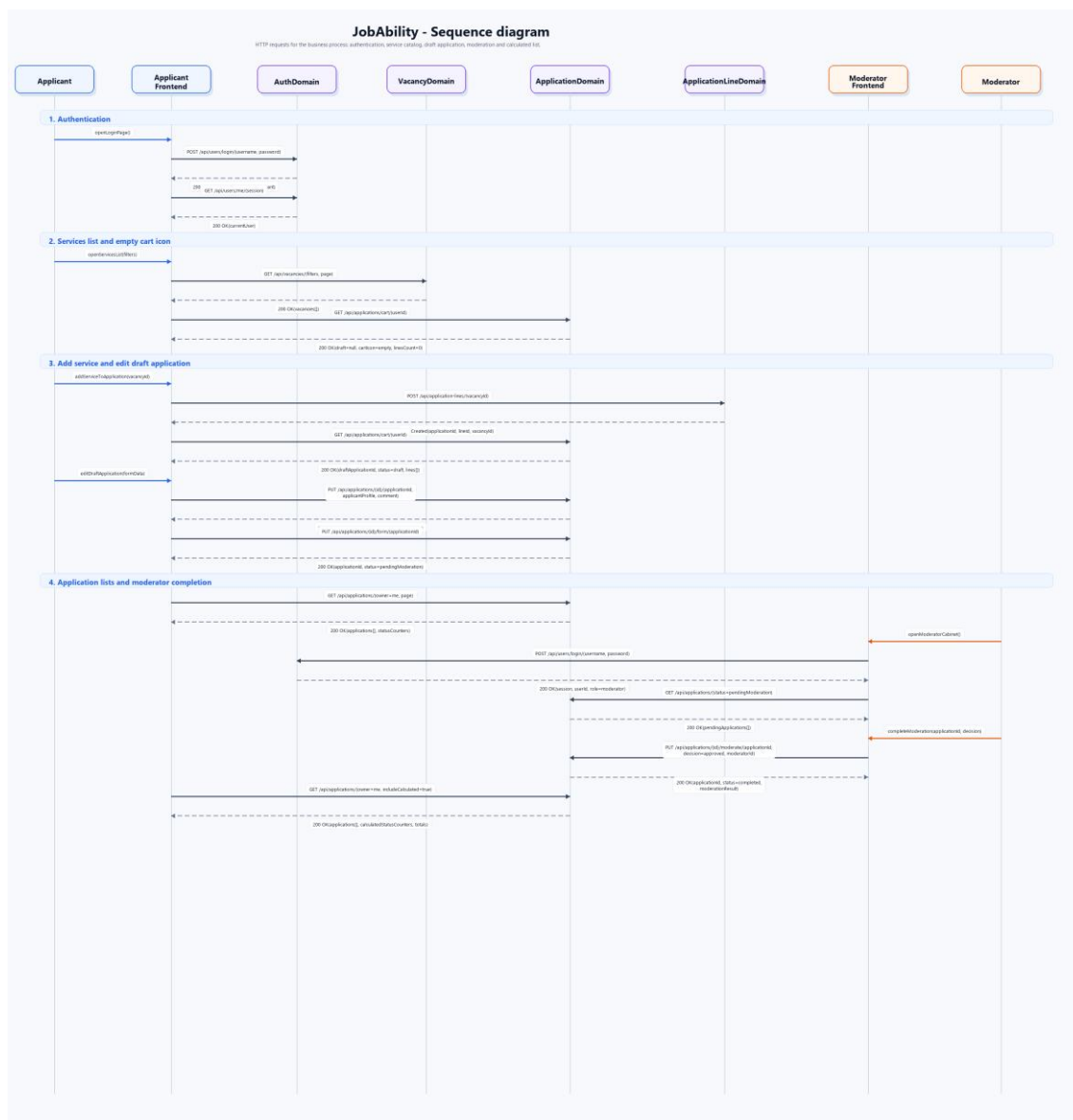


Рисунок 7 — Диаграмма последовательности HTTP-запросов

Описание интерфейса

Интерфейс JobAbility реализован как адаптивное SPA-приложение. Навигационная панель содержит переходы на главную страницу, каталог вакансий, Swagger, административные разделы и действия входа/регистрации. На мобильных экранах блоки перестраиваются вертикально, карточки вакансий занимают всю ширину, а формы сохраняют крупные поля ввода.

Главная страница объясняет назначение сервиса: система трудоустройства для людей с ограниченными возможностями. Страница регистрации позволяет выбрать роль соискателя или работодателя, указать

имя, фамилию, username, email и пароль. Страница входа выполняет авторизацию и переводит пользователя в сценарий, соответствующий его роли.

Каталог вакансий отображает карточки с названием, компанией, городом, зарплатой, графиком и описанием поддержки доступности. Карточка вакансии показывает подробное описание и позволяет добавить вакансию в заявку. Кабинет соискателя содержит профиль и переход к списку заявок. Страница заявки позволяет редактировать строки, комментарии, порядок и формировать заявку.

Кабинет работодателя используется для создания вакансий и просмотра статуса модерации. Страница откликов работодателя показывает заявки, в которых присутствуют его вакансии. Кабинет модератора содержит список вакансий на проверке и список заявок; модератор может одобрять или отклонять вакансии, завершать или отклонять заявки.

Таблица 6 — Окна интерфейса и действия пользователей

Окно	Маршрут	Действия	API
Главная	/	Переход к каталогу, входу, регистрации	GET /
Регистрация	/register	Создание аккаунта соискателя или работодателя	POST /api/users/register/
Вход	/login	Авторизация и восстановление сессии	POST /api/users/login/, GET /api/users/me/
Вакансии	/vacancies	Поиск, фильтрация, открытие карточек	GET /api/vacancies/
Карточка вакансии	/vacancies/:id	Просмотр и добавление в заявку	GET /api/vacancies/{id}/, POST /api/application-lines/
Заявки	/applications	Список, фильтры, расчет сумм	GET /api/applications/
Детальная заявка	/applications/:id	Редактирование, формирование, модерация	GET/PUT /api/applications/{id}/
Кабинет соискателя	/cabinet/applicant	Профиль, пароль, переход к заявкам	GET/PUT /api/users/profile/
Кабинет работодателя	/cabinet/employer	Создание вакансий и просмотр статусов	POST /api/vacancies/, GET /api/vacancies/mine/
Отклики работодателя	/cabinet/employer/responses	Просмотр заявок по вакансиям работодателя	GET /api/applications/employer-responses/
Кабинет модератора	/cabinet/moderator	Модерация вакансий и заявок	GET /api/vacancies/pending/, PUT

			/moderate/
Swagger	/swagger/	Просмотр API-документации	GET /swagger/

Заключение

В результате работы разработана и развернута система JobAbility — web-сервис трудоустройства для людей с ограниченными возможностями. Реализованы роли гостя, соискателя, работодателя и модератора; каталог вакансий; карточка вакансии; профиль соискателя; черновик и формирование заявки; модерация вакансий и заявок; кабинеты пользователей.

Подготовлена инфраструктура: Docker-образы backend и frontend, Kubernetes-манифесты для frontend, backend, PostgreSQL, Redis, MinIO, Prometheus, Grafana и Adminer, Ingress через Traefik, PVC для постоянных данных, Secret и ConfigMap. Настроен GitLab Runner с shell executor и pipeline build-test-upload-deploy. Тестовая стадия запускает Django-тесты перед доставкой.

Итоговый результат подтвержден успешным pipeline и работающим приложением на виртуальной машине. Проект опубликован в GitLab: <https://gitlab.com/xflame0xx1/jobability>. Демо через временный Cloudflare Tunnel использовалось для проверки публичного доступа; локальный адрес стенда: <http://192.168.56.19/>.

Список использованных источников

1. World Health Organization. Disability: fact sheet. URL: <https://www.who.int/news-room/fact-sheets/detail/disability-and-health> (дата обращения: 29.05.2026).

2. International Labour Organization. Disability inclusion at work. URL: <https://www.ilo.org/topics/disability-and-work> (дата обращения: 29.05.2026).

3. Федеральный закон от 24.11.1995 № 181-ФЗ «О социальной защите инвалидов в Российской Федерации». URL:

https://www.consultant.ru/document/cons_doc_LAW_8559/ (дата обращения: 29.05.2026).

4. Kuznetsov O., Melchiori M., Frontoni E., Arnesano M. A Production-Ready Machine Learning System for Inclusive Employment: Requirements Engineering and Implementation of AI-Driven Disability Job Matching Platform. arXiv:2508.11713, 2025. URL: <https://arxiv.org/abs/2508.11713>.

5. Kamruzzaman M., Kim G. L. The Impact of Disability Disclosure on Fairness and Bias in LLM-Driven Candidate Selection. arXiv:2506.00256, 2025. URL: <https://arxiv.org/abs/2506.00256>.

6. Soiffer N. et al. Barriers to Employment: The Deaf Multimedia Authoring Tax. arXiv:2505.01030, 2025. URL: <https://arxiv.org/abs/2505.01030>.

7. Django Software Foundation. Django documentation 5.2. URL: <https://docs.djangoproject.com/en/5.2/>.

8. Django REST framework. API Guide. URL: <https://www.django-rest-framework.org/api-guide/>.

9. React documentation. URL: <https://react.dev/>.

10. Kubernetes Documentation. Concepts: workloads, services, ingress. URL: <https://kubernetes.io/docs/concepts/>.

11. Docker Documentation. Build and Dockerfile reference. URL: <https://docs.docker.com/build/>.

12. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/>.

13. GitLab Documentation. CI/CD pipelines. URL: <https://docs.gitlab.com/ci/>.

14. MinIO Documentation. URL: <https://min.io/docs/minio/kubernetes/upstream/>.

15. Redis Documentation. URL: <https://redis.io/docs/latest/>.

16. Google Patents. Patent publication US20140279637A1 and patent search results for employment matching systems. URL: <https://patents.google.com/>.

17. HeadHunter. URL: <https://hh.ru/>.

18. Портал «Работа России». URL: <https://trudvsem.ru/>.

19. AbilityJobs. URL: <https://www.abilityjobs.com/>.

Приложение А. Техническое задание

1. Цель разработки — создать итоговую систему JobAbility для поддержки трудоустройства людей с ограниченными возможностями, включая публикацию доступных вакансий, формирование заявок, модерацию и демонстрационное развертывание в Kubernetes.

2. Назначение системы — предоставить гостям публичный каталог вакансий, соискателям инструмент подбора и оформления заявки, работодателям кабинет публикации вакансий, модератору инструмент контроля качества данных и завершения заявок.

3. Задачи разработки: 3.1 реализовать backend-домены Auth, Vacancy, Application, Profile на Django 5.2 и DRF; 3.2 реализовать REST API с OpenAPI/Swagger; 3.3 реализовать роли и session-based authentication; 3.4 реализовать React/Vite frontend с адаптивными страницами; 3.5 реализовать черновик заявки и расчет total_salary; 3.6 реализовать кабинеты работодателя и модератора; 3.7 добавить Django-тесты и CI test stage; 3.8 контейнеризовать backend/frontend и развернуть в k3s; 3.9 подготовить нативный сценарий/desktop-демо на базе web-интерфейса при необходимости; 3.10 подготовить демо GitHub Pages для статического описания проекта; 3.11 оформить документацию, диаграммы, Swagger и РПЗ; 3.12 вести репозиторий Git/GitLab с pipeline build-test-upload-deploy.

4. Функциональные требования. 4.1 HTTP-методы системы должны включать GET, POST, PUT и DELETE для доменов Auth, Vacancy, Profile, Application, ApplicationLine, EmployerResponses и System. 4.2 Меню должно содержать переходы на главную страницу, вакансии, Swagger, вход, регистрацию, кабинеты и заявки в зависимости от роли. 4.3 Гость должен иметь страницу регистрации с вызовом POST /api/users/register/. 4.4 Гость и пользователь должны иметь страницу аутентификации с POST /api/users/login/ и GET /api/users/me/. 4.5 Гость должен видеть список

услуг/вакансий через GET /api/vacancies/. 4.6 Гость или соискатель должен видеть одну услугу/вакансию через GET /api/vacancies/{id}/, а соискатель должен добавлять ее в заявку через POST /api/application-lines/. 4.7 Создатель заявки должен иметь страницу заявки и список заявок с GET /api/applications/, GET/PUT /api/applications/{id}/, PUT /api/applications/{id}/form/. 4.8 Модератор должен иметь дополнительные функции списка заявок и вакансий: GET /api/vacancies/pending/, PUT /api/vacancies/{id}/moderate/, PUT /api/applications/{id}/moderate/.

5. Требования к аппаратному обеспечению сервера: минимум 2 vCPU, 6-8 GiB RAM, 23 GiB диска для демонстрационного стенда; для роста медиафайлов предусмотреть отдельный PVC MinIO. Клиент: любой ПК или смартфон с современным браузером, экраном от 360 px по ширине и доступом к HTTP/HTTPS.

6. Требования к программному обеспечению сервера: Ubuntu 22.04 LTS, Docker Engine 29.2.1, k3s v1.35.5+k3s1, containerd 2.2.3-k3s1, GitLab Runner 19.0.0, Python 3.13, Django 5.2, Django REST Framework, PostgreSQL, Redis, MinIO, Nginx, Prometheus, Grafana, Adminer. Клиент: Chrome/Edge/Firefox актуальной версии, поддержка JavaScript ES2022 и CSS Grid/Flexbox.

Приложение Б. Методы веб-сервиса

Домен Auth.

Таблица Б.1 — Методы домена Auth

№	Метод	URL	Описание	Входные данные	Выходные данные
1	GET	/api/users/me/	Текущий пользователь	-	id:int, username:str, role:str, email:str
2	POST	/api/users/register/	Регистрация	username:str, password:str, first_name:str, last_name:str, email:str, role:str	User
3	POST	/api/users/login/	Вход	username:str, password:str	User + session
4	POST	/api/users/logout/	Выход	-	message:str
5	PUT	/api/users/password/	Смена пароля	old_password:str, new_password:str	message:str

Домен Vacancy.

Таблица Б.2 — Методы домена Vacancy

№	Метод	URL	Описание	Входные данные	Выходные данные
6	GET	/api/vacancies/	Список вакансий	search:str, city:str, salary:int	Vacancy[]
7	POST	/api/vacancies/	Создание вакансии	title, company, city, salary, description, image:file	Vacancy
8	GET	/api/vacancies/{id}/	Карточка вакансии	id:int	Vacancy
9	GET	/api/vacancies/mine/	Мои вакансии	-	Vacancy[]
10	GET	/api/vacancies/pending/	Вакансии на модерации	-	Vacancy[]
11	PUT	/api/vacancies/{id}/moderate/	Модерация вакансии	action:str, note:str	Vacancy

Домен Profile.

Таблица Б.3 — Методы домена Profile

№	Метод	URL	Описание	Входные данные	Выходные данные
12	GET	/api/users/profile/	Профиль	-	ApplicantProfile
13	PUT	/api/users/profile/	Обновить профиль	full_name, phone, city, age, gender, disability_category	ApplicantProfile

Домен Application.

Таблица Б.4 — Методы домена Application

№	Метод	URL	Описание	Входные данные	Выходные данные
14	GET	/api/applications/cart/	Пустая/текущая корзина	-	application_id:int null, items_count:int
18	GET	/api/applications/	Список заявок	status:str, date_from:date	Application[]
19	GET	/api/applications/{id}/	Детали заявки	id:int	ApplicationDetail
20	PUT	/api/applications/{id}/	Редактировать заявку	profile/contact fields	ApplicationDetail
21	PUT	/api/applications/{id}/form/	Сформировать заявку	id:int	ApplicationDetail
22	PUT	/api/applications/{id}/moderate/	Завершить/отклонить	action:finish reject, moderator_note:str	ApplicationDetail
23	DELETE	/api/applications/{id}/delete/	Удалить черновик	id:int	204 No Content

Домен ApplicationLine.

Таблица Б.5 — Методы домена ApplicationLine

№	Метод	URL	Описание	Входные данные	Выходные данные
15	POST	/api/application-lines/	Добавить вакансию	vacancy_id:int, qty:int	ApplicationLine
16	PUT	/api/application-lines/	Изменить строку	vacancy_id:int, qty:int, comment:str, is_main:bool	ApplicationLine

17	DELETE	/api/application-lines/	Удалить строку	vacancy_id:int	204 No Content
----	--------	-------------------------	----------------	----------------	----------------

Домен Employer.

Таблица Б.6 — Методы домена Employer

№	Метод	URL	Описание	Входные данные	Выходные данные
24	GET	/api/applications/employer-responses/	Отклики работодателя	-	Application[]

Домен System.

Таблица Б.7 — Методы домена System

№	Метод	URL	Описание	Входные данные	Выходные данные
25	GET	/api/schema/	OpenAPI schema	-	OpenAPI JSON
26	GET	/swagger/	Swagger UI	-	HTML
27	GET	/metrics/	Метрики Prometheus	-	text/plain